

Laboratory Manual

Course: **Algorithm Analysis and Design Lab**

Experiment Title: **Greedy-Based Multi-Booth Job Scheduling System Using C Language**

1. Experiment Information

Item	Details
Course Title	Algorithm Analysis and Design Lab
Experiment No.	1
Experiment Title	Greedy-Based Customer Job Scheduling
Programming Language	C
Algorithm Technique	Greedy Approach
Recommended IDE	Code::Blocks / Dev-C++ / Visual Studio Code
Submission Type	Source Code + Report + Output Screenshot

2. Objective of the Experiment

The objectives of this laboratory experiment are:

- To understand the concept of the **Greedy Algorithm**.
- To implement a real-life scheduling system using greedy strategy.
- To simulate a multi-service booth customer handling environment.
- To calculate customer waiting times.
- To analyze booth utilization efficiency.
- To calculate:
 - Average waiting time of customers
 - Total service period of each booth
 - Idle time of each booth
- To develop problem-solving skills using C programming.

3. Introduction

In many real-world systems such as banks, hospitals, ticket counters, customer care centers, and supermarkets, customers arrive at different times and wait for available service booths.

Efficient scheduling is necessary to minimize customer waiting time and improve service quality.

One common approach to solve this type of problem is the **Greedy Algorithm**. A greedy algorithm makes the best immediate decision at each step without reconsidering previous decisions.

In this experiment, students will develop a customer scheduling system where:

- Multiple service booths are available.
- Customers arrive at different times.
- Each customer requires a certain service duration.
- The system assigns customers to booths using a greedy strategy.

The booth that becomes available first should receive the next customer.

The program will also evaluate the performance of the scheduling system by calculating various metrics.

4. Problem Description

A service center contains:

- 3 service booths
- At least 20 customers

Each customer has:

1. Customer ID
2. Arrival Time
3. Service Time

Customers arrive at different times.

Example:

Customer Arrival Time

C1	1
C2	3
C3	4
C4	4
C5	8

The system must assign customers to booths dynamically using a greedy approach.

5. Greedy Strategy Used

The greedy rule for this experiment is:

Assign the next customer to the booth that becomes available earliest.

This approach attempts to:

- reduce waiting time
- increase booth utilization
- reduce booth idle time

6. Functional Requirements

Input Requirements

The program should take:

- Number of customers (minimum 20)
- Arrival time of each customer
- Service time of each customer

Processing Requirements

The program should:

1. Track availability of all booths
2. Assign customers greedily
3. Calculate:
 - Service start time
 - Service completion time
 - Waiting time
4. Track booth busy and idle periods

Output Requirements

The program must display:

Customer Information Table

Customer	Arrival	Service	Booth	Start	Finish	Waiting
----------	---------	---------	-------	-------	--------	---------

Booth Statistics

For each booth:

- Total service time
- Total idle time

Overall Statistics

- Average waiting time
- Total simulation time

7. Important Scheduling Concepts

7.1 Arrival Time

The time at which a customer enters the system.

Example:

Customer C3 arrives at minute 4

7.2 Service Time

The amount of time required to serve a customer.

Example:

Customer C3 requires 5 minutes

7.3 Waiting Time

The amount of time a customer waits before service begins.

Formula:

Waiting Time = Service Start Time - Arrival Time

7.4 Completion Time

The time when customer service finishes.

Formula:

Completion Time = Start Time + Service Time

7.5 Idle Time

The period when a booth remains free without serving customers.

Example:

If Booth 1 finishes at minute 10 and the next customer arrives at minute 14:

Idle Time = 14 - 10 = 4 minutes

8. Example Dataset

Students may use the following sample input.

Customer	Arrival Time	Service Time
C1	1	4
C2	3	5
C3	4	2
C4	4	3
C5	8	6
C6	9	2
C7	10	5
C8	11	4
C9	13	3
C10	15	7
C11	17	5
C12	18	2
C13	20	4
C14	21	3
C15	23	6
C16	24	4
C17	26	5
C18	27	3
C19	30	2
C20	32	4

9. Suggested Algorithm

Step-by-Step Procedure

Step 1

Initialize:

- 3 booths
- Booth available times = 0

Step 2

Read customer information.

Step 3

For each customer:

- Find booth with minimum available time
- Assign customer to that booth

Step 4

Determine:

$\text{Start Time} = \max(\text{Arrival Time}, \text{Booth Available Time})$

Step 5

Calculate:

$\text{Waiting Time} = \text{Start Time} - \text{Arrival Time}$

Step 6

Calculate completion time:

$\text{Completion Time} = \text{Start Time} + \text{Service Time}$

Step 7

Update booth availability.

Step 8

Repeat for all customers.

10. Pseudocode

```
Initialize boothAvailable[3] = {0,0,0}

For each customer
{
    Find booth with minimum available time

    startTime = max(arrivalTime, boothAvailable)

    waitingTime = startTime - arrivalTime

    finishTime = startTime + serviceTime

    Update boothAvailable = finishTime
}
```

11. Sample Output Format

Customer Scheduling Table

Customer	Arrival	Service	Booth	Start	Finish	Waiting
C1	1	4	B1	1	5	0
C2	3	5	B2	3	8	0
C3	4	2	B3	4	6	0
C4	4	3	B1	5	8	1

Booth Statistics

Booth	Service Time	Idle Time
Booth 1	25	4
Booth 2	28	3
Booth 3	24	5

Overall Statistics

Average Waiting Time = Total Waiting Time / Number of Customers

Example:

Average Waiting Time = 18 / 20 = 0.9 minutes

12. Expected Learning Outcomes

After completing this experiment, students will be able to:

- Understand greedy scheduling techniques
- Simulate real-life service systems
- Analyze scheduling performance
- Implement scheduling algorithms in C
- Calculate waiting and idle times
- Compare booth utilization efficiency

13. Complexity Analysis

Students should analyze:

Time Complexity

If there are:

- n customers
- b booths

Then:

Time Complexity = $O(n \times b)$

Since booths are fixed (3 booths):

$O(n)$

Space Complexity

Arrays are used for storing customer information.

Space Complexity = $O(n)$

14. Additional Challenge Tasks (Optional)

Students may additionally implement:

- Dynamic number of booths
- Priority customers
- Random customer generation
- Booth utilization percentage

15. Conclusion

This experiment demonstrates how greedy algorithms can efficiently solve real-world scheduling problems. By assigning customers to the earliest available booth, the system minimizes waiting time and improves service utilization.

The experiment also helps students understand important software engineering concepts such as:

- scheduling
- resource allocation
- performance analysis
- simulation systems
- algorithm efficiency

These concepts are widely used in operating systems, cloud computing, banking software, customer support systems and distributed applications.